A
Project Report
On

Decentralized Cloud Storage Using Mobile Devices with Encrypted Chunk Distribution and Incentive Mechanism

Submitted in the fulfilment of the requirement of the

degree of

Bachelor of Technology

In

COMPUTER SCIENCE & ENGINEERING

submitted by:

Rudra Pratap Singh (2023022014)
Vanshika Srivastava (2023022017)
Vishal Tiwari (2023022018)

under the supervision of

Mr. Manish Kumar Gupta
Assistant Professor, CSE Department

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MADAN MOHAN MALAVIYA UNIVERSITY OF TECHNOLOGY
GORAKHPUR-273010

DECLARATION

We hereby declare that this project submission is our original work and that, to the best of our knowledge and belief, it does not contain any material previously published or written by another person, nor does it contain any material that has been accepted for the award of any other degree or diploma at any university or institution of higher learning, except where due acknowledgment has been made in the text.

Sign:

Student Name: Rudra Pratap Singh

Roll No: 2023022014

Date:

Place: Gorakhpur

Sign:

Student Name: Vanshika Srivastava

Roll No: 2023022017

Sign:

Student Name: Vishal Tiwari

Roll No: 2023022018

CERTIFICATE

This certifies that the project entitled **"Decentralized Cloud Storage Using Mobile Devices with Encrypted Chunk Distribution and Incentive Mechanism"** was completed by the students **Rudra Pratap Singh (Roll No. 2023022014), Vanshika Srivastava (Roll No. 2023022017), and Vishal Tiwari (Roll No. 2023022018)** of the **Bachelor of Technology in the Department of Computer Science & Engineering** at Madan Mohan Malaviya University of Technology, Gorakhpur, in partial fulfilment of the requirements for their degree.

Signature of Mentor

Mr. Manish Kumar Gupta
Assistant Professor,
Department of CSE
MMMUT Gorakhpur

Signature of HOD

Dr. Rakesh Kumar
Head of Department,
Department of CSE
MMMUT Gorakhpur

Date:

Place: Gorakhpur

APPROVAL SHEET

This project report entitled **"Decentralized Cloud Storage Using Mobile Devices with Encrypted Chunk Distribution and Incentive Mechanism"** by Rudra Pratap Singh, Vanshika Srivastava and Vishal Tiwari is approved for the degree for Bachelor of Technology in Computer Science and Engineering.

Examiner:

_____

Supervisor:

_____

Mr. Manish Kumar Gupta

Head of Department:

_____

Dr. Rakesh Kumar

Date:

Place: Gorakhpur

ACKNOWLEDGEMENT

We wish to express our profound gratitude to all those who have supported the development of project **"Decentralized Cloud Storage Using Mobile Devices with Encrypted Chunk Distribution and Incentive Mechanism."**

Our sincere thanks are extended to our project guide, **Mr. Manish Kumar Gupta** sir, for their invaluable mentorship and insightful critiques throughout this endeavour. We are also thankful to the faculty of the Compuuter Science and Engineering at Madan Mohan Malaviya University of Technology for providing a strong academic foundation.

Finally, we express our sincere gratitude to the almighty, our parents, and our friends for their continuous encouragement, moral support, and motivation. Without their unwavering support, this project would not have been possible.

ABSTRACT

Traditional cloud storage systems depend more on data centers and they have to spend a lot of money on infrastructure. They also have problems with growing data. They are at risk of things like data breaches and service outages. At the time there are billions of smartphones all over the world that have a lot of unused storage space. This paper talks about a kind of cloud storage system that uses smartphones to store data. This system is cheaper can grow easily and is more reliable [4]. The system breaks down files into encrypted pieces using the AES algorithm. It then sends these pieces to different mobile devices. The system decides which devices to use based on things like how storage space they have their battery level, how well they can process information and how stable their network connection is [12]. Also, a metadata server that keeps track of where all the pieces are stored. This makes it easy and secure to get the data back. The proposed system makes sure that data is always available. It also makes the system more resilient to problems [7]. It does not depend as much on big data centers. The system also has a way to reward users for participating. This makes the cloud storage system sustainable, secure and good, for large-scale use. The cloud storage system is a solution because it uses smartphones as storage nodes. The cloud storage system is cost-effective and scalable. The cloud storage system is also resilient and secure.

List of Figures

S.No	Description of Figure	Page no.
1.	System Architecture	9
2.	Data Upload Flow	10
3.	Data Retrieval Flow	11
4.	System Failover Mechanism	12
5.	Incentive Model	13
6.	File Upload Time Analysis	20
7.	File Download Time Analysis	20
8.	Cost Analysis	21
9.	Decryption Speed Comparison	22
10.	Encryption Payload	22
11.	Security Analysis	23
12.	Device Health Scoring Analysis	24

List of Tables

S.No	Description of Figure	Page no.
1.	Comparative Analysis of Existing Research Work	4

Table of Content

Declaration i

Certificate ii

Approval Sheet iii

Acknowledgment iv

Abstract v

List of Figures vi

List of Tables vii

Chapters

1. Introduction 1
- 1.1 Introduction
- 1.2 Motivation
- 1.3 Contribution

2. Literature Review 3
- 2.1 Overview of Existing Systems
- 2.2 Related Research Work
- 2.3 Research Gap of the Proposed System
- 2.4 Problem Statement

3. System Design and Architecture 7
- 3.1 Proposed System Overview
- 3.2 System Architecture
- 3.3 System Components
- 3.4 Working Principle
- 3.5 Design Key Features
- 3.6 System Flowcharts

4. Methodology 15
- 4.1 Overview
- 4.2 Step by Step Methodology

5. Results and Analysis 19
- 5.1 Overview
- 5.2 Expected System Behavior
- 5.3 Performance Evaluation
- 5.4 Conversation
- 5.5 Performance Evaluation Through Graphs and Charts

6. Conclusion & Future Scope 25
- 6.1 Conclusion
- 6.2 Future Scope

References viii

Chapter 1
Introduction

1.1 Introduction

Nowadays, dependence on digital applications has increased rapidly, so cloud storage has become an essential part for individuals and organisations. However, traditional cloud storage platforms heavily rely on centralised data centres, controlled by a few providers. These data centres are very costly and require a high amount of energy. Simultaneously, there is an intense amount of smartphones available with unused storage that remains idle for a very long time. To tackle this, this project titled "Decentralised Cloud Storage Using Mobile Devices with Encrypted Chunk Distribution and Incentive Mechanism," proposes an effective way to use smartphones as cloud storage nodes. This system is not only reliable, with the use of techniques like file chunking, encryption, and distribution across multiple nodes, but is also very cost-effective as compared to conventional platforms. The incentive model motivates users to contribute their unused storage by rewarding them, and charges the renters based on space usage.

1.2 Motivation

With the technological advancements, digital applications have become a crucial part of our lives, and these applications require cloud storage to efficiently store, manage, and access user data remotely, with high convenience. Whereas, currently, all the major cloud service providers rely on data centres that are not only expensive but also energy-intensive, with only a few service providers. This system also has limited user control and scalability limitations. Centralised systems are also prone to high security risk because a single system failure, cyberattack, or data breach could affect millions of users.

On the other hand, there are billions of smartphones with a high amount of unused storage, and this storage remains idle for a very long time. These devices could be utilised to build decentralised storage networks. This project aims to build a decentralised cloud storage system using smartphone nodes, where the user data will be stored in multiple replicas in the form of encrypted chunks. Replication leads to data availability, even if some of the devices go offline, and encryption ensures data confidentiality.

The system reduces dependency on centralised, expensive cloud storage platforms and also improves f lexibility by removing the single point of failure and making the system more scalable as more users join. Additionally, the incentive model creates a sustainable cooperative ecosystem. Overall, the main aim of this project is to reduce the centralised dependency with a cost-effective, secure, scalable and reliable cloud alternative.

1.3 Contribution

This project makes the following contributions to the development of decentralized cloud storage systems:

.

1. Decentralised storage systems will reduce the dependence on centralised data centres.

2. Chunking the file and distributing it across multiple nodes will increase security and confidentiality.

3. Multiple replicas are stored, and an emergency backup system is used to improve reliability
.
4. Smart node selection based on battery levels, storage space, and network conditions increases the system performance.

5. Users are rewarded for providing the unused storage, which contributes to high user participation.

Chapter 2

Literature Review

2.1 Overview of Existing systems

Traditional cloud providers that are available in the market, such as Amazon S3, Google Cloud, and Microsoft Azure, use centralised data centres to store and access data. These providers provide high security, confidentiality, easy storage and management; however, they often require high computation cost, large infrastructure, and complex management. Due to centralised systems, these platforms are also prone to data breaches, service outages, and cyberattacks. To tackle these problems, some decentralised cloud systems are also available in the market, such as IPFS, Filecoin, and Storj [1], [2]. These systems distribute data peer to peer instead of centralised access. These systems increase the fault tolerance and reduce centralised dependency, but it often requires high computational cost, high energy consumption, and blockchain integrations [1], [10].

2.2 Related Research Work

Protocol Labs et al. (2020) - Filecoin introduced decentralized storage using cryptographic Proof-of-Replication and Proof-of-Spacetime mechanisms to guarantee verifiable and reliable storage. Their model emphasized incentive-driven participation by rewarding storage providers, establishing an economically sustainable decentralized marketplace [1].

Benet & Greco et al. (2020) - IPFS expanded distributed storage through enhanced content addressing and deduplication mechanisms. Their work tackled redundancy and link durability using content hashing and Merkle-DAG structures, enabling inore resilient decentralized applications, although lacking a built-in incentive layer [2].

Wang et al. (2021) Hybrid Mobile Edge Storage proposed a framework in which smartphones collaboratively stored encrypted file fragments while edge servers handled metadata. Their architecture improved availability during mobility and unstable connectivity [3].

Zhang et al. (2021) Energy-Aware Peer Storage introduced battery-sensitive scheduling techniques for allocating storage tasks. Their system reduced power consumption by nearly 32% while maintaining peer reliability in mobile clusters [4].

Jangid et al. (2022) Incentive Cloud Storage developed a decentralized credit-reward framework to motivate participation. Their findings showed improved storage availability through fair and dynamic incentives [5].

Park et al. (2022) Secure Multi-Node Storage implemented AES-GCM chunk-level authenticated encryption and multi-replica strategies, enhancing integrity and survivability during partial node failures while introducing processing overhead on resource-constrained devices [6].

Raman et al. (2023) - Smartphone Cluster Storage integrated erasure coding in crowdsourced mobile storage clusters and demonstrated strong fault tolerance even when up to 40% of nodes became unavailable [8].

Sato et al. (2024) - Chunk-Level Encryption for Edge Devices proposed nonce-rotated AES-GCM encryption and authenticated metadata, enhancing resistance against replay attacks in highly dynamic environments [9].

Al-Harbi et al. (2024) Blockchain-Based Incentive Marketplace introduced a transparent credit distribution model using blockchain, improving trust and fairness but adding transaction cost and delay [10].

Kwan et al. (2025) - Self Healing Mobile Storage Grid implemented background repair jobs for automated replication restoration, proving effective in unstable mobile environments [11].

Ortega et al. (2025) - AI-Based Chunk Placement used machine-learning node scoring based on battery, bandwidth, and reliability, improving retrieval success by nearly 27% in dynamic networks [12].

Table 2.1 Comparative Analysis of Existing Research Work

S.No.	Research Name	Methods Used	Limitations	Research Gap
1.	Protocol Labs et al. (2020) – Filecoin	Proof-of-Replication, Proof- of-Spacetime, incentive-driven decentralized storage	High computational cost; requires powerful nodes	Efficient lightweight consensus for mobile/low-power storage nodes
2.	Benet & Greco et al. (2020) – IPFS	Content addressing, Merkle-DAG, distributed hash tables	No native incentive model; replication left to applications	Integration of incentive layers + reliable mobile-node replication
3.	Wang et al. (2021) – Hybrid Mobile-Edge Storage	Phone-based chunk storage + edge metadata server	Metadata centralization weakens decentralization	Fully decentralized metadata storage suitable for mobile devices
4.	Zhang et al. (2021) – Energy-Aware Peer Storage	Battery-aware chunk scheduling, adaptive device selection	Lower redundancy in low-energy conditions	Balanced trade-off between energy- saving and fault tolerance
5.	Jangid et al. (2022) – Incentive Cloud Storage	Credit-reward incentive system, participation-based scoring	Complex reward fairness; possible credit abuse	Secure, automated incentive auditing using cryptographic proofs

6.	Park et al. (2022) – Secure Multi-Node Storage	AES-GCM chunk encryption, multi- replica storage	High CPU overhead on mobile nodes	Lightweight encryption/authentic ion optimized for smartphones
7.	Hernandez et al. (2023) – Redundancy-Optimized Mobile Storage	Adaptive replication using device health metrics	Requires continuous monitoring; energy- intensive	Low-overhead monitoring methods that reduce device usage
8.	Raman et al. (2023) – Smartphone Cluster Storage	Erasure coding, crowdsourced device clusters	Heavy encoding/deco ding overhead	Efficient erasure codes for mobile/low-power hardware
9.	Sato et al. (2024) – GCM Nonce-Rotation Storage	Nonce rotation, authenticated chunk metadata	Requires secure key distribution	Decentralized key management without centralized KMS
10.	Al-Harbi et al. (2024) – Incentive Marketplace	Blockchain-based credit accounting	Blockchain latency and cost	Faster low-cost distributed ledgers for mobile storage systems
11.	Ortega et al. (2025) – AI-Based Chunk Placemen	ML-based device scoring (battery, stability, bandwidth)	Requires model retraining; high data dependency	Federated learning for decentralized, privacy-preserving optimization

2.3 Research Gap of the Proposed System

Although there are various decentralized models available that address challenges such as redundancy, security, and energy efficiency, a unified framework that uses smartphone-based storage, secure chunk-encryption, fault tolerance, and an incentive model is still lacking [8]. Modern solutions either depend on blockchain technologies or half centralized systems. As a result, there is still requirement for a lightweight, smartphone-based decentralized storage that efficiently ensures security, reliability and scalability.

2.4 Problem Statement

The proposed system solves the following problems:

1. It solves the dependency on centralized cloud storage because current cloud solutions depend heavily on centralized data centers, charging high cost, central point of failure and many problems.

2. It also solves the problem of unused storage capacity in smartphones.

3. It solves the challenges of fault tolerance and data availability.

4. The system must check mobile nodes health status such as battery level, storage availability, and network strength to select it.

5. Continuous communication and data transfer may decrease the mobile battery and resources.

Chapter 3

System Design and Architecture

3.1 Proposed System Overview

The proposed system is trying to move from traditional centralized data centers. In this system everything relies on devices. These days mobile devices have become storage nodes and they create a peer-to-peer network [3], [8]. There is no server that can control the entire system. For example, when a user is uploading a file the proposed system breaks the file into chunks. Then each chunk is encrypted using techniques and distributed across multiple mobile devices [6]. The system does not randomly choose the devices to store the chunks. Instead it checks the battery level of the devices and the available storage space and then it selects the mobile devices to store the chunks. To prevent data loss the system stores copies of each chunk across multiple mobile devices [7], [8]. If some mobile devices go offline the data is still safe because the system has a metadata management unit that keeps track of the devices and the chunks they store. This makes the data retrieval process very efficient for the users of the devices. The proposed system also has an incentive model that rewards the devices that contribute more to the system [5], [10]. This means that the mobile devices that store chunks and make more resources available to the system get rewarded more, than the mobile devices that contribute less to the proposed system [7], [8].

3.2 System Architecture

The architecture is divided into 5 layers that work together to store data and efficiently on mobile devices. The first layer is client layer which behaves as an interface for users to interact with the system. This handles user authentication, file uploads and downloads, and communicate with backend. The user interface is an app that people use. The processing layer manages operations such as file segmentation, encryption and device selection [6], [12]. The storage layer consists of distributed mobile devices. Each device stores encrypted data chunks in its memory and continuously shares its health status [3], [4]. The metadata layer stores information such as chunk identifiers, storage locations, and redundancy mappings [3]. This layer is very beneficial in data retrieval if some nodes are unavailable. To make the system more reliable, a backup layer is available that sores data copies in a stable environment [7]. This acts as a safeguard for our system. These layers execute sequentially, where user requests pass from the client layer to the processing layer, and then to storage and metadata layers for execution and tracking.

3.3 System Components

Client Layer acts as an interface between the user and the system and allows users to upload and download files, basic authentication, and transfers user requests to the backend for processing.

Processing Layer handles core file operations such as file segmentation, encryption, and selection of storage devices [6], [12]. It ensures that data is fully prepared and distributed efficiently. **Storage Layer** consists of distributed mobile devices that store encrypted data chunks of file. Each device contributes storage space and shares its current health status, such as battery level and storage availability [3], [4]. Metadata Layer maintains information about file chunk their locations, and redundancy mappings [3]. It enables fast and accurate data retrieval without storing the actual data centrally. **Backup Layer** ensures reliability by storing copies of important data in a stable storage environment [7], [8]. It ensures data recovery in case of multiple node failures.

3.4 Working Principle

By coordinating processes across distributed mobile devices, the system functions [3], [7]. When a user uploads a file, the backend divides it into small sections. As a next step the system encrypts each section with AES encryption. In a neutral assessment, the system checks every smartphone for the amount of energy in its battery. It also measures the amount of storage that remains empty. There is a check for the intensity of the network signal. For those reasons, the system assigns sections to nodes that have the highest level of functionality [4], [12]. To manage the system, the central database stores metadata about where sections are located. On this database there are also details about encryption. And the database holds records of the copies that exist for redundancy. During retrieval the server identifies where sections are - using the metadata table. Then it gets the encrypted sections from the mobile nodes. It checks if the data is accurate. By using a key the server decrypts the data. It then combines the sections into the original file. If a section is missing on a device, the system uses a copy from another node. With those redundant copies, the system completes the process to build the file again. If many devices fail at the same time, the system gets the file from the emergency static storage. To ensure safety it sends sections to new nodes. In this way the state of redundancy is present again [11].

3.5 Design Key Features

- **Chunk Storage Security:** Automation in file splitting and encryption with AES for safety across mobile devices [6], [9].
- **Redundancy:** Multiple replicas of each chunk prevent data loss during node failures [7], [11].
- **Smart Device Selection:** Real-time Battery, Storage, and Network checks choose the best nodes [4], [12].
- **Fast Retrieval:** Efficient metadata management enables quick chunk lookup and file reconstruction [3].
- **Scalability:** The storage capacity automatically grows as more smartphone users join the network [7].
- **Failover Support:** Emergency storage prevents data loss [11].

3.6 System Flowcharts

3.6.1 System Architecture

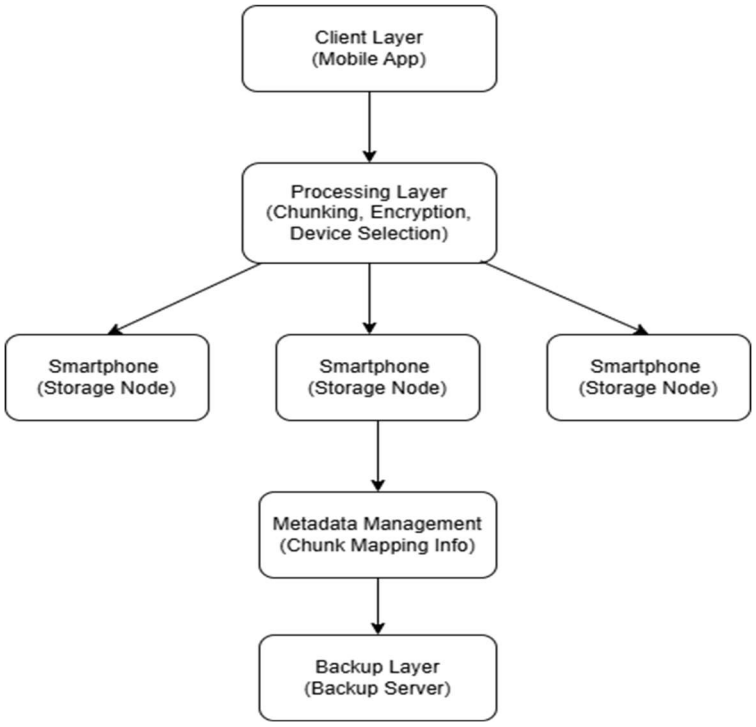

Fig 1: System Architecture

The proposed system in Fig. 1 represents a decentralized mini cloud storage architecture that utilizes smartphones as storage nodes to provide a secure and scalable storage solution. The system begins with the client layer, implemented as a mobile application, through which users interact with the platform to upload, access, or manage their data.

Once a file is uploaded, it is processed in the processing layer, where it is divided into smaller chunks, encrypted to ensure data security, and suitable storage devices are selected based on predefined criteria. These chunks are then distributed across multiple smartphones, improving data availability, fault tolerance, and efficient utilization of unused storage space.

The metadata management layer plays a crucial role by maintaining information about chunk locations, enabling efficient data retrieval. Additionally, a backup layer ensures reliability by storing copies of data on a backup server. This architecture ensures security, scalability, and reliability in data storage and retrieval.

3.6.2 Data Upload Flow

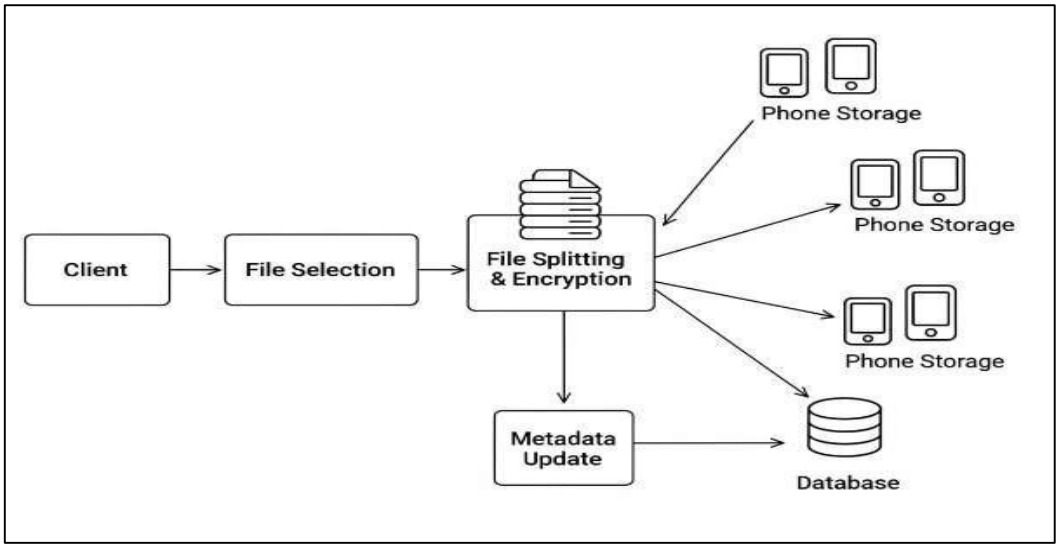

Fig 2: Data Upload Flow

1. Client & File Selection

- The client authenticates and selects a file to upload.
- Unique file ID and hash, such as SHA-256 for integrity checking.
- A symmetric key for encryption is generated for the file.

2. File Splitting

- The file is divided into smaller pieces.
- Every chunk is labeled with an index and offset for proper reassembly.

3. Encryption of Chunks

- Each chunk is independently encrypted using the file's encryption key.
- Authentication tags and a unique nonce are added for data integrity.
- Chunk hashes are computed for verification.

4. Distribution to Storage Nodes

- The encrypted chunks are sent to a set of phone storages—what we'll refer to as nodes.
- Depending on the policy, it can either replicate chunks or store with erasure coding.
- This ensures redundancy: even if some devices fail, the file remains recoverable.

5. Metadata Management

- File ID, file size, total chunks
- Chunk IDs, storage node mappings, encryption metadata
- Key reference (but not the plaintext key).
- The database serves as an index for retrieval.

6. Retrieval Process

- When a client requests the file, the system fetches the needed chunks from the storage nodes.
- Hash and authentication tag are checked chunks are decrypted and reassembled in order.
- The final file is checked against the original hash for integrity.

7. Fault Tolerance & Recovery

- **Primary method:** The system switches to other available replicas when the retrieval from the main storage nodes fails.
- **Secondary method:** If the cloud storage and the replicas are unavailable, the system retrieves data from a static database holding backup copies for emergency access.
- Background repair jobs maintain redundancy by creating new replicas when necessary.

8. Security Measures

- End-to-end encryption ensures confidentiality: nodes just store ciphertext.
- Key management protects file keys.
- Communication with storage nodes is secured using TLS.

3.6.3 Data Retrieval Flow

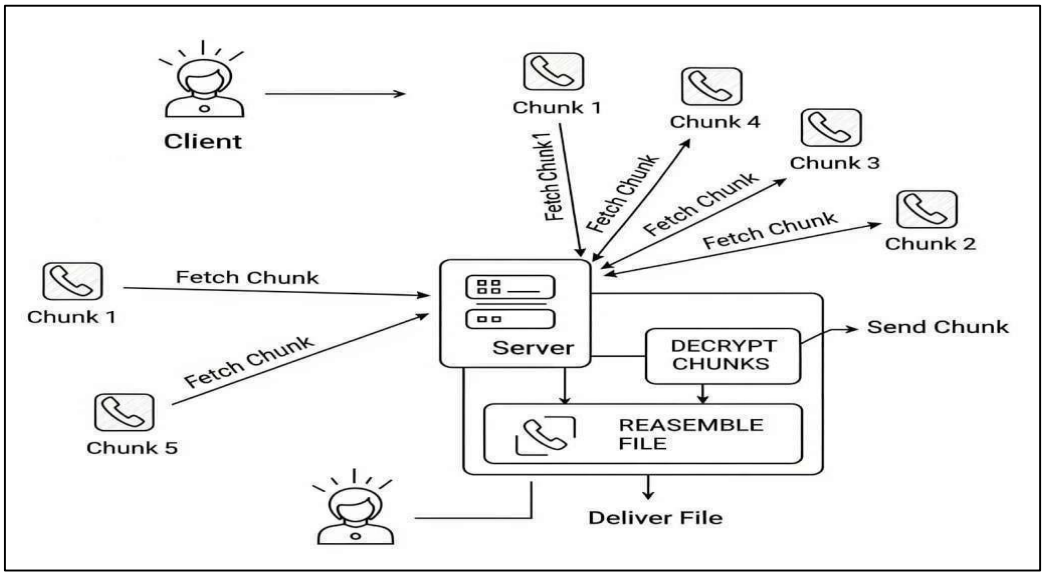

Fig 3: Data Retrieval Flow

1) **Request Initiation:** The client request for the download of the previously uploaded file.
2) **Server Handling:** The server receives the request and begins processing.

1) **Metadata Lookup:** The server queries the PostgreSQL metadata database to identify chunk IDs, device mappings, and storage locations.
2) **Chunk Retrieval:** The server retrieves the encrypted chunks from mobile devices based on metadata.
3) **Redundancy Check:** If any chunk is missing or corrupted, then its availability is ensured by using the redundant copies.
4) **Decryption & Reassembly:** Retrieved chunks are decrypted and reassembled into the original file at the server.
5) **Delivery:** The reconstructed file is securely delivered to the requesting client (renter).

3.1.2 System Failover & Emergency Static Storage

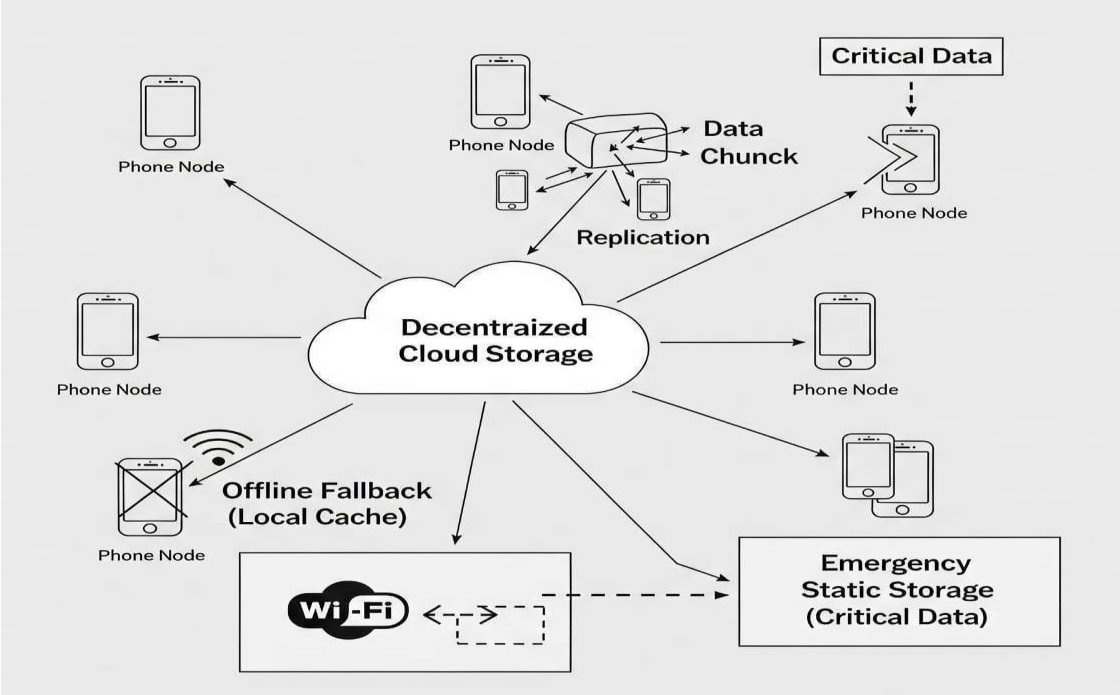

Fig 4: System Failover Mechanism

Case A: 1–2 Chunks Missing

1. The backend fetches all chunks from the lenders.
2. If some chunks are missing (node offline, data lost):
   - The backend checks whether replicas are available on the other lender devices.
   - Missing chunks are fetched from replicas.
   - File is successfully reassembled.

Result: Retrieval succeeds due to redundancy.

Case B: The Majority of Nodes Fail (But Not All)

1. If many nodes fail but at least one replica of each chunk exists → backend still reassembles file.
2. The system marks failed nodes as "unreliable" in metadata.
3. Backend automatically creates new replicas on other active nodes when they come automatically so that in the future reliability is guaranteed.

Result: File restored from remaining nodes + self-healing replication.

Case C: All nodes fail completely

1. Cannot pull any chunk from the backend to lender devices.
2. Backend triggers emergency delivery mechanism:
   - It retrieves the file directly from static storage; that is, an emergency backup.
   - Serves file to renter.
3. When some lender devices come online again → backend redistributes new chunks from static backup to restore redundancy.

Result: File still delivered but from the static storage, not distributed nodes.

3.1.3 Incentive Model

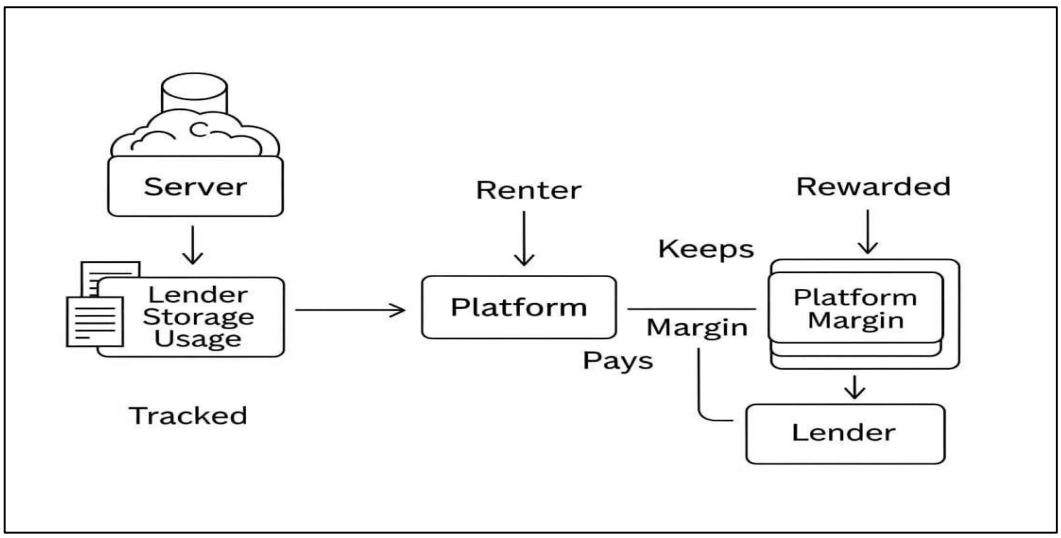

Fig 5: Incentive Model

1. Lender → Platform (Storage Supply & Tracking)

- A lender provides unused storage space to the platform.
- A server constantly monitors the lender's storage usage, how much space is provided, and how actively it is in use, whether in real time or seldom.
- Tracking is necessary as the incentive of a lender depends on type and level of usage.

2. **Renter → Platform (Storage Demand & Pricing)**

   Each time a renter comes onto the platform requesting storage, the platform analyzes:
   - Real-time / high-availability demand (fast, frequently accessed).
   - Rare / archival demand (infrequently accessed).
   - The platform allocates storage from suitable lenders:
   - Active lenders → real-time usage.
   - Less-active lenders → rare storage.
   - The renter pays the platform based on the kind of usage: real-time storage costs more; rare usage costs is cheaper.

4. **Platform Margin (Profit Mechanism)**

   The platform holds a margin of what the tenant pays. From the remaining amount, it rewards the lender:
   - Higher incentive → if their space is used regularly in real time.
   - Less incentive → if there is less utilization of their space.
   - Your profit = Money charged to renter – Incentive paid to lender.

Chapter 4

Methodology

4.1 Overview

The methodology proposed for the decentralized cloud storage system spells out the structured steps followed to design and implement a secure, distributed, and smartphone- driven storage framework. The core objective of this system is to transform ordinary mobile devices into reliable storage nodes, safely store encrypted pieces of files across them, and ensure seamless data retrieval even when multiple nodes failure [3], [7].

4.2 Step-by-Step Methodology

Step 1: System Initialization and Device Registration:
Initially, the backend system initialized modules such as encryption, device monitoring, and metadata management. Mobile nodes register with the system and continuously share their health status, such as battery level, storage availability, and network conditions. Only healthy nodes are selected for participation based on predefined criteria [4], [12].

Step 2: File upload Process: This phase allows users to upload files through the mobile application. The request gets sent to the backend server for processing, and a unique file ID is generated.

Algorithm 1: File Upload Process
Input: File F, chunk size k
Output: Distributed encrypted chunks across storage nodes

1. Generate a unique file identifier (file_id)
2. Divide file F into chunks of size k → {c1, c2, ..., cn}
3. for each chunk ci do
    a. Compute hash hi for integrity verification
    b. Encrypt ci using AES → ei
  end for
4. Select suitable storage nodes S using device selection criteria
5. for each encrypted chunk ei do
    a. Store ei on selected node(s)
    b. Create replicas for redundancy
    c. Update metadata server with:
      (file_id, chunk_index, node_location, hash)
  end for
6. return success status

Step 3: Chunking and Encryption: In this phase, the uploaded file is divided into smaller chunks. Each chunk is assigned a unique index and a hash value to verify integrity. These chunks are then encrypted using AES encryption to ensure confidentiality before distribution [6].

Algorithm 2: File Chunking and Encryption

Input: File F, chunk size k

Output: Encrypted chunks C

```
Divide file F into chunks of size k
for each chunk ci do
   generate a hash hi for integrity
Encrypt CI using AES
   store encrypted chunk
end for
return set of encrypted chunks C
```

Step 4: Device Selection: In this phase, the system selects suitable mobile devices based on their health conditions, such as battery percentage, storage availability, and network stability. Devices that do not meet the required criteria are not selected for storage [4], [12].

Algorithm 3: Device Selection

Input: Set of available devices D

Output: Selected devices S

```
Initialize S ← ∅
for each device d ∈ D do
  if d.battery ≥ threshold_battery AND
     d.storage ≥ required_storage AND
     d.network is stable then
        S ← S ∪ {d}
  end if
end for
return S
```

Step 5: Data Distribution: In this phase, encrypted data chunks are distributed across multiple selected mobile devices. Each chunk is stored on multiple devices to ensure redundancy. The metadata server keeps track of chunk locations, replica details, and associated identifiers for efficient tracking [3], [7].

Algorithm 4: Data Distribution with Redundancy
Input: Set of encrypted chunks C, set of selected devices D, replication factor r
Output: Distributed chunks with metadata entries

```
for each chunk ci ∈ C do
 selected_nodes ← choose r devices from D
   for each device d ∈ selected_nodes do
       store ci on device d
       update metadata server
   end for
end for
return success
```

Step 6: Redundancy Management and Monitoring: In this phase, the proposed system continuously monitors device availability and data integrity. If a device becomes unavailable or a data chunk becomes unavailable, new replicas are automatically created on other healthy devices. This ensures continuous data availability and fault tolerance [7], [8].

Algorithm 5: Redundancy Management
Input: Stored chunks C, device status
Output: Updated replicas

```
for each chunk ci do
  if replica count < required replicas OR device fails then
      select new healthy device
      create new replica of ci
  end if
end for
```

Step 7: Data Retrieval and Reconstruction: When a user requests a file, the system searches for the chunk location in the metadata server. Then the required chunks are fetched from devices where chunks are stored and integrity of chunks is verified using hash values. Then chunks are decrypted and in the correct order to reconstruct the original file [3], [6].

Algorithm 6: File Reconstruction
Input: Set of encrypted chunks C
Output: Reconstructed file F

```
for each chunk ci ∈ C do
  verify integrity using hash
  decrypt ci
end for
merge all chunks in correct order
return reconstructed file F
```

Step 8: Failover and Backup Recovery: The proposed system contains a failover method to handle the situation of devices failure. If chunks are missing they are recovered from replicas or retrieved from backup server based on missing level and later redistributes chunks when devices reconnect. This ensures data availability and reliability [7], [11].

Algorithm 7: Failover and Backup Recovery

Input: metadata M, device status D, replication factor r

Output: Restored chunk availability

```
for each chunk ci in metadata M do
  available_nodes ← devices storing ci that are active
  if count(available_nodes) < r then
     if replicas exist then
        select healthy device d_new
        replicate ci from available_nodes to d_new
        update metadata M
     else
        retrieve ci from backup server
        store ci on selected healthy devices
        update metadata M
     end if
  end if
end for
return system consistency maintained
```

Step 9: Failover and Backup Recovery: Incentive Mechanism: The proposed system gives incentives to devices based on factors such as storage usage and frequency of data access. This helps in making profit for both lender and renter [5], [10].

Algorithm 8: Incentive Calculation

Input: Set of devices D

Output: Reward score for each device

```
for each device d ∈ D do
  storage_score ← amount of storage contributed by d
  access_score ← frequency of data access handled by d
  reward(d) ← α * storage_score + β * access_score
  update reward balance for device d
end for
return reward distribution
```

Chapter 5

Results and Analysis

5.1 Overview

The decentralized storage system aims to provide seamless storage services by using smartphones as storage nodes [3], [8]. It processes the data files and divides them into chunks, followed by distributing them across multiple nodes. It aims to achieve high availability of data by performing multiple integrity and security checks on nodes during the retrieval operation. The architecture ensures smooth connectivity between backend, metadata servers, and storage nodes. Additionally, it also provides an incentive model for encouraging user participation [5], [10].

5.2 Expected System Behavior

The system will continuously perform automated operations to ensure efficient data upload and retrieval. During data upload, data files will be divided into chunks, and each chunk will be encrypted with the AES encryption technique, and distributed to the best suitable nodes based on the storage level, battery level and network conditions [4], [6], [12]. During data retrieval, the system will check for data integrity, confidentiality, and reassemble the chunks into the original data. If the data part is found as encrypted or missing, it will retrieve the data from alternative replicas, or from the emergency backup if all the available nodes fail [7], [8].

5.3 Performance Evaluation

- **Redundancy Success Rate:** Since the data is distributed across multiple nodes, it ensures high availability [7], [8].
- **Retrieval Time Efficiency:** The system ensures high data upload and retrieval time under normal conditions.
- **Data Security Strength**: AES-encrypted chunks increase the security of nodes [6].
- **Fault Tolerance Rate:** Multiple nodes and emergency backups make the system highly fault-tolerant [7], [11].
- **Scalability Growth:** The storage capacity increases with the growing number of users, thus promising high scalability [3].

5.4 Conversation

The main idea of the system is to use smartphones as storage nodes, which could possibly increase the flexibility of the system to fulfil user demands. Further, chunking the data files, storing multiple copies, and having an emergency backup for the worst case, makes the system secure, highly available and fault-tolerant [7]. The health-based device selection also increases the performance of nodes. An incentive-based model is another noticeable point to motivate users to use their vacant storage in return [5].

5.5 Performance Evaluation Through Graphs and Charts

File Upload Time Analysis:

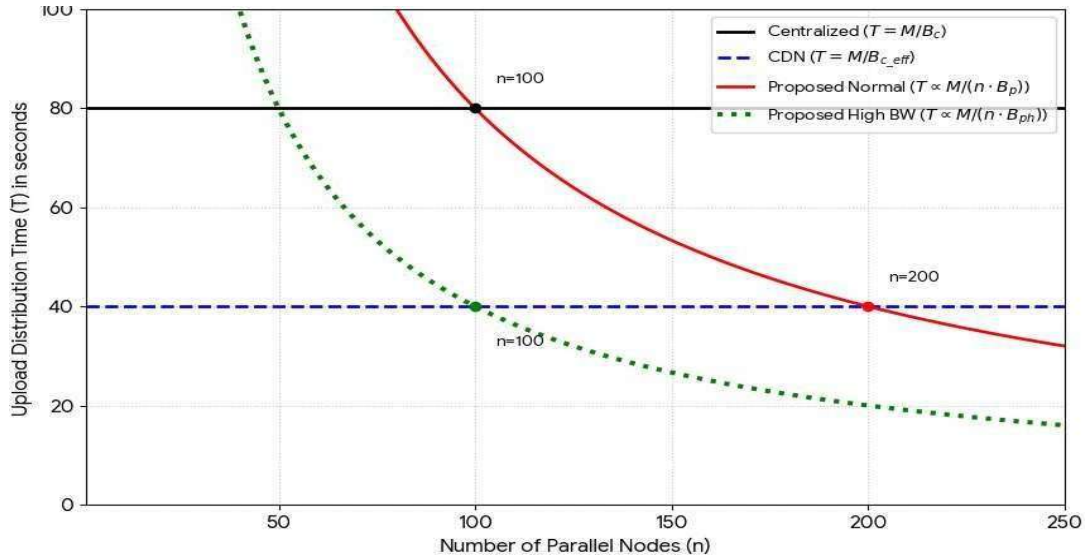

Fig 6: File Upload Time Analysis

The Fig. 6 shows how upload time decreases as the number of parallel nodes increases for the proposed system, while remaining constant for centralized and CDN-based models due to fixed bottlenecks. The proposed approach benefits from parallelism, where more nodes lead to faster distribution. Under high-bandwidth conditions, the improvement is even more significant, achieving the lowest upload times.

File Download Time Analysis:

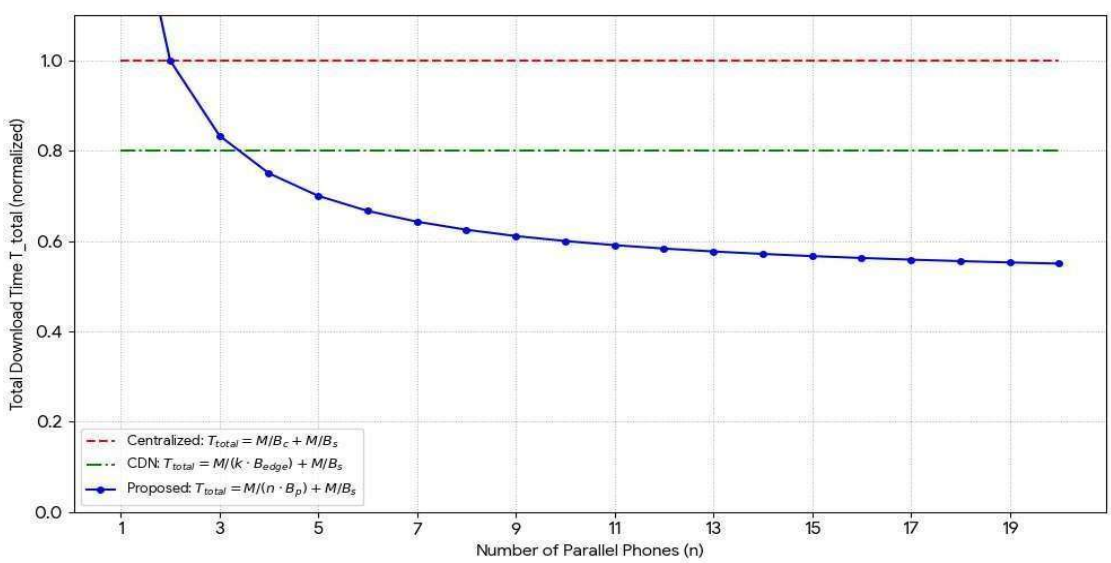

Fig 7: File Download Time Analysis

This Fig. 7 shows that as the number of parallel phones increases, the total normalized download time in the proposed system decreases rapidly due to parallel data fetching. In contrast, the centralized model remains fixed at the highest time and the CDN model stays constant at a lower level, reflecting infrastructure limits. The proposed approach continuously improves with more devices, eventually outperforming both traditional models. This demonstrates the efficiency and scalability of cooperative, decentralized downloading.

Cost Analysis:

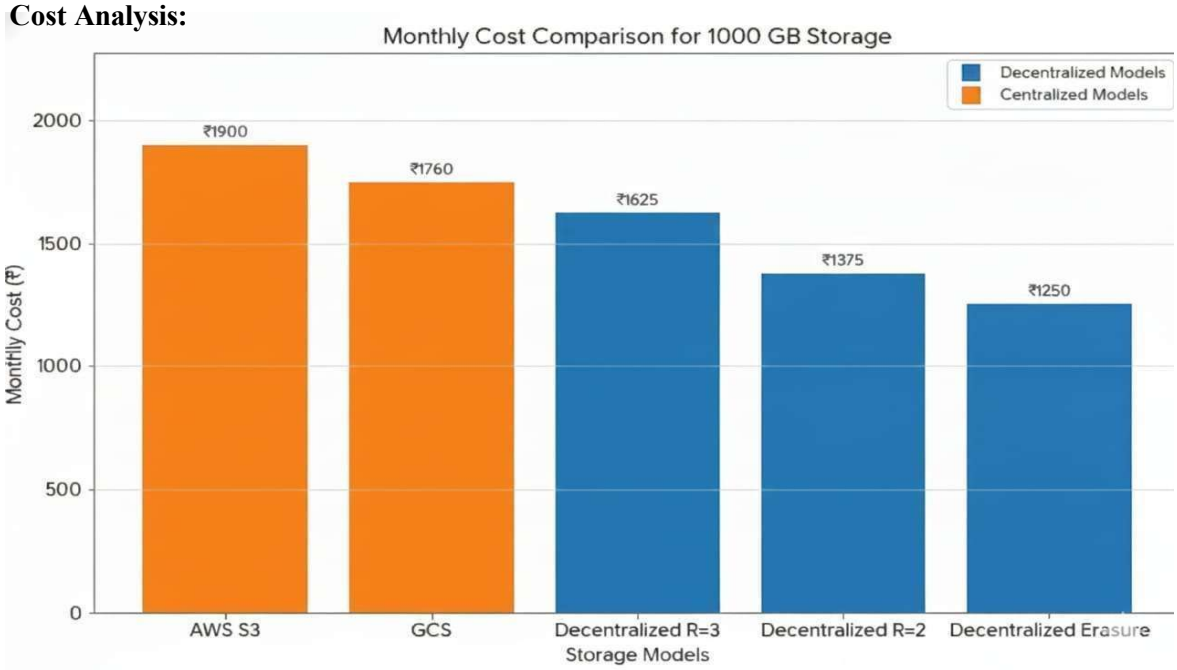

Fig 8: Cost Analysis

The bar chart in Fig. 8 presents a comparison of the monthly expenses for 1000 GB of data storage when users choose between centralized cloud services and decentralized models. It is clear that centralized platforms like AWS S3 require higher payments. On the contrary decentralized models that use different redundancy schemes offer prices that are lower. As developers optimize redundancy - using erasure coding, the cost to store data decreases further. In this way decentralized storage is more economical. By looking at the graph, it is evident that decentralized systems provide a financial advantage for large scale data storage. When compared to traditional cloud storage, those systems are cheaper.

.

Decryption Speed Comparison:

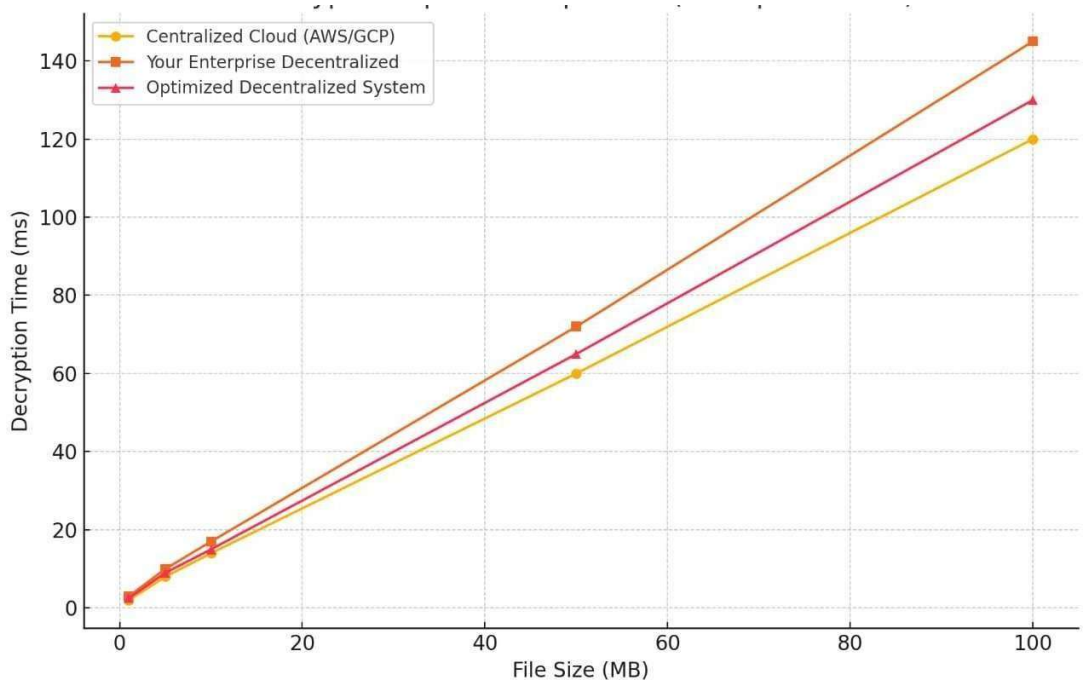

Fig 9: Decryption Speed Comparison

The graph in Fig. 9 shows that decryption time increases as file size grows for all systems. Centralized cloud performs the fastest, while the basic decentralized system is slightly slower due to processing overhead. The optimized decentralized system significantly reduces this delay, achieving performance close to centralized platforms. This proves that proper optimization makes decentralized storage both efficient and practical.

Encryption Overhead:

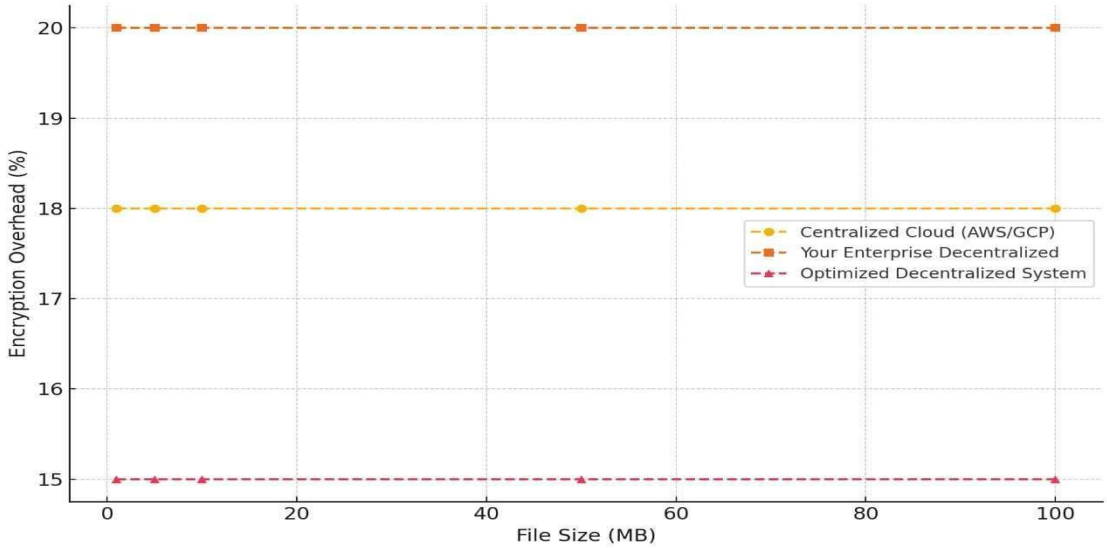

Fig 10: Encryption Overhead

As data points illustrate in Fig. 10, the time required for encryption is the same for every file size in each environment. By comparison the decentralized system for large organizations requires the most extra processing time. In a different manner, the central server cluster occupies a median position for those requirements. Then again the decentralized system with enhanced efficiency uses the smallest amount of extra processing time. There is evidence that the improvement process decreases the resource use of encryption. If those changes occur, the ability of the system to handle larger workloads remains unchanged.

Security Analysis:

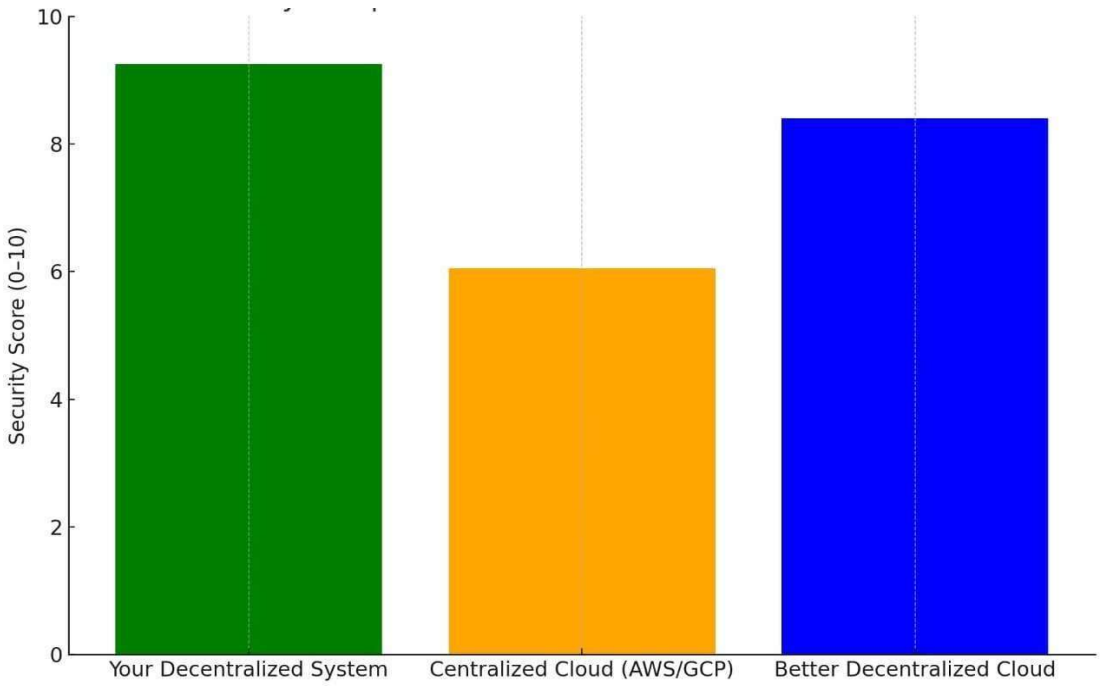

Fig 11: Security Analysis

The Fig. 11 displays how security levels differ across storage methods. It is clear that the suggested decentralized system protects data most effectively. By comparison the updated decentralized cloud reaches a medium level of safety - but the centralized cloud protects information the least. There is a clear advantage when a system uses many nodes to store data. As an example the system encodes information into a format that unauthorized users cannot read. On another level the network spreads data across different locations. Because of this design, the system functions even if one part stops working. To secure data those architectures remove the need for a single control point.

Device Health Scoring (Battery, Storage, Network):

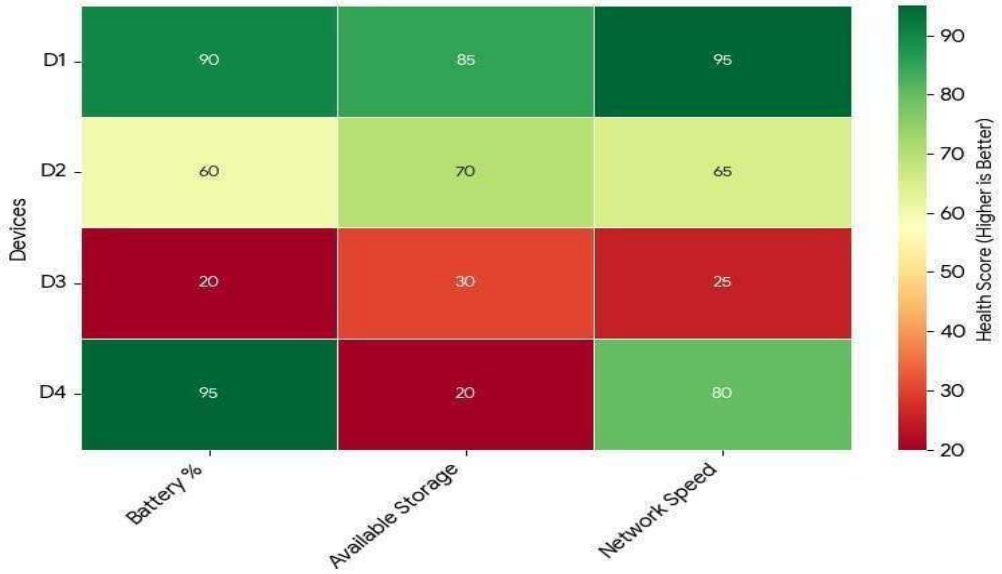

Fig 12: Device Health Scoring Analysis

As this visualization illustrates in Fig. 12, the system evaluates how storage devices perform. It determines status - looking at the amount of power in a battery. It also looks at the space that is open for files. It looks at how fast data moves across a network. There are higher values marked in green for hardware that functions without failure. In this state the system puts pieces of data on those nodes first - but there are lower values marked in red for equipment that lacks stability. To prevent errors, the system does not use those areas. On a constant basis, the software tracks every machine. By doing this the system chooses nodes that work well. Because of this selection, the storage process remains fast. With this method the data stays safe.

Chapter 6
Conclusion and Future Scope

6.1 Conclusion

Instead of relying on expensive centralized data servers, the project presents a secure, scalable, and cost-effective method to use mobile devices as storage nodes. It offers security, availability, confidentiality, and reliability in case of node failure with the help of file chunking, AES encryption, redundancy, and health-based device selection. The fair model inspires users to use their unused space and get paid in return [10]. It creates a sustainable environment for the users to rent and lend data. The emergency backup recovery system makes the architecture highly fault-tolerant [7], [11c]. With continued development, the system has a high potential to replace the conventional data centre and become a cost-effective, secure and reliable option in future.

6.2 Future Scope

For future work, there could be an automated AI-driven system to split the data into chunks [12]. Real-time health monitoring of the storage node will increase the system's performance and reliability on the nodes. Advanced encryption techniques like zero-knowledge encryption could increase the security and confidentiality. Storage replication could be replaced by erasure coding to optimize storage space [8]. With these increased integrations, the architecture can evolve into a global, low-cost cloud alternative that provides security, confidentiality, data availability, and scalability for digital appliances.

References

1. Protocol Labs, “Filecoin: A Decentralized Storage Network,” Protocol Labs Research Report, 2020.
2. J. Benet and D. Greco, “IPFS: Content-Addressed, Peer-to-Peer File System for Decentralized Applications,” Protocol Labs Technical Whitepaper, 2020.
3. Y. Wang, H. Chen, and Z. Li, “A Hybrid Mobile-Edge Storage Model for Distributed Data Fragmentation,” IEEE Transactions on Mobile Computing, vol. 20, no. 5, pp. 1882–1895, 2021.
4. L. Zhang and R. Kumar, “Energy-Aware Peer Storage Using Mobile Devices in Edge Environments,” IEEE Access, vol. 9, pp. 112040–112053, 2021.
5. A. Jangid and R. Patel, “Incentive-Based Storage Networks Using Dynamic Credit-Reward Mechanisms,” Journal of Network and Computer Applications, vol. 197, pp. 103289, 2022.
6. S. Park and P. Thapa, “Authenticated Chunk Encryption and Multi-Replica Storage for Secure Distributed Systems,” IEEE Communications Letters, vol. 26, no. 3, pp. 611–615, 2022.
7. M. Hernandez, T. Alvarez, and G. Prieto, “Redundancy-Optimized Replication for Mobile Cloud Storage,” Future Generation Computer Systems, vol. 144, pp. 327–339, 2023.
8. S. Raman and A. Bose, “Distributed Storage Using Smartphone Clusters with Erasure Coding,” IEEE Transactions on Cloud Computing, vol. 11, no. 2, pp. 499–512, 2023.
9. K. Sato and H. Lin, “Secure Distributed Chunk Encryption Using GCM Nonce Rotation for Edge Storage,” IEEE Internet of Things Journal, vol. 11, no. 1, pp. 845–857, 2024.
10. A. Al-Harbi and S. Singh, “Blockchain-Based Incentive Marketplace for Decentralized Edge Storage Networks,” IEEE Transactions on Services Computing, vol. 17, no. 4, pp. 912–925, 2024.
11. T. Kwan and M. Priyadarshi, “Self-Healing Storage Grids for Mobile Device Networks,” ACM Journal on Autonomous and Adaptive Systems, vol. 20, no. 2, pp. 1–15, 2025.
12. R. Ortega and F. Malik, “AI-Driven Dynamic Chunk Placement with Device Reliability Scoring,” IEEE Transactions on Emerging Topics in Computing, vol. 13, no. 2, pp. 278–290, 2025.